

Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software – Appello del 10 luglio 2014

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Se il compito risultasse insufficiente, non sarà possibile partecipare al prossimo appello.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 5:

Esercizio 1

É data la specifica della classe `IntSet` (come presentata in aula), con i seguenti metodi:

- osservatori puri `isIn`, `size`;
- costruttori: `IntSet`;
- modificatori: `insert`, `remove`.

Si vuole aggiungere un nuovo metodo, chiamato `extract`, che restituisce un IntSet e che ha parametro `x` intero. Il metodo solleva un'eccezione `IllegalArgument` se `x` non appartiene all'insieme corrente, altrimenti restituisce il sottoinsieme dei valori contenuti nell'insieme corrente inferiori a `x`.

1. Fornire una specifica formale del metodo extract, ipotizzando che si tratti di un osservatore puro.
2. Definire un insieme di casi di test per il metodo utilizzando un approccio *black-box*
3. Supporre che il metodo `extract` abbia l'effetto collaterale di eliminare dall'insieme corrente il sottoinsieme calcolato. Come cambia la specifica?
4. Come cambia l'insieme dei dati di test per la nuova versione del metodo?

SOLUZIONE
Punto 1
IL metodo extract deve essere dichiarato puro e deve avere questa clausola ensures

```
//@ ensures (forall int i;;(result.isIn(i) <==> this.isIn(i) && i<x));
//@signals (IllegalArgumentException e) (!isIn(x));
public /*@ pure @*/ IntSet extract(int x);
```

Punto 2

1. caso in cui x non appartiene e insieme non vuoto, p.es. $(3, \{1, 5, 2\})$, risultato atteso: $\{1, 5, 2\}$.
2. caso in cui insieme vuoto, p.es. $(3, \{\})$, risultato atteso: $\{\}$
3. caso in cui x appartiene (e ovviamente insieme non vuoto), p.es. $(3, \{1, 3, 2\})$, risultato atteso: $\{1, 2\}$
4. caso in cui il risultato è vuoto, p.es. $(3, \{3, 5, 6\})$, risultato atteso: $\{\}$
5. caso in cui il risultato è il massimo sottoinsieme che può essere ritornato, cioè è l'intero insieme tranne il valore massimo, p.es. $(3, \{1, 3, 2\})$, risultato atteso: $\{1, 2\}$

Punto 3
Il metodo non è piu' puro. Nella specifica di ensures cui al punto 1 a this bisogna sostituire old(this) e aggiungere la condizione che tutti e soli gli elementi diversi da x sono ancora nell'insieme. Anche la signals deve modificata: il metodo non è puro quindi occorre imporre che this non cambi.

```
\begin{verbatim}
//@ ensures (forall int i;;result.isIn(i) <==> this.isIn(i) && i<x) &&
//@   (forall int i;;this.isIn(i) <==> \old(this.isIn(i)) && i>=x);
//@signals (IllegalArgumentException e) (!isIn(x)) &&
//@  (forall int i;;isIn(i) <==> \old(isIn(i)));
public IntSet extract(int x);
```

Punto 4
Se non era presente nei test definiti per la versione pura del metodo (ma nel nostro caso lo era), aggiungere un test in cui vengono eliminati zero elementi e uno in cui vengono eliminati tutti gli elementi. Il metodo non è puro, quindi i test precedenti devono anche verificare il valore di this al termine della chiamata (i valori di this sono rispettivamente: $\{1, 5, 2\}, \{\}, \{\}, \{5, 6\}, \{\}$.

Esercizio 2

Si considerino le seguenti classi Java:

```
public class Frutto {
   public String toString(){return "Frutto";}
   public void macedonia(Frutto f){System.out.println(this + " - " + f);}
   public void confronta(Frutto f){System.out.println("Sono uguali");}
}

public class Mela extends Frutto {
   public String toString(){return "Mela";}
   public void macedonia(Mela f){System.out.println(this + " - " + f);}
   public void confronta(Mela m){System.out.println("Sono diversi");}
}

public class Arancia extends Frutto {
   public String toString(){return "Arancia";}
   public void macedonia(Arancia f){System.out.println(this + " - " + f);}
   public void confronta(Arancia a){System.out.println("Non ho idea");}
}
```

e si spieghi cosa stamperebbe il seguente metodo `main`, motivando brevemente le risposte:

```
public class Test {
   public static void main(String args[]) {
      Frutto f1, f2;
      Mela m;
      Arancia a;

      f1 = new Mela();
      f2 = new Arancia();
      m = new Mela();
      a = new Arancia();

      f1.macedonia(f2);
      f1.confronta(f2);


      a.confronta(f2);
      a.confronta(m);
      f2 = m;

      f1.macedonia(f2);
      a.macedonia(m);
      a.confronta(a);
   }
}
```

SOLUZIONE
Le stampe sono
Mela-Arancia
Sono uguali
Sono uguali
Sono uguali
Mela-Mela
Arancia-Mela
non ho idea

Esercizio 3

É data la specifica della classe `Punto` con i seguenti metodi:

- osservatori puri (getter) `float getX()`, `float getY()` per ottenere le coordinare X e Y in un piano;
- costruttore `Punto(float x, float y)`.

Si vuole specificare una classe `NuvolaPunti` con i seguenti metodi:

- osservatori puri `Punto getBaricentro()`,`float getPrecisione()`, `int numPunti()`, `boolean isIn(Punto p);`
- costruttore `NuvolaPunti(float baricentro, float precisione);`
- modificatori `inserisciPunto(Punto p) throws violaInvariante, eliminaPunto(Punto p) throws violaInvariante`.

Sostanzialmente la classe rappresenta un insieme di punti (`numPunti` fornisce la cardinalità dell'insieme) che devono rispettare un vincolo dato da un invariante (vedi punto successivo), la cui potenziale violazione viene segnalata da una eccezione da parte dei metodi modificatori.

1. Si vuole che la classe soddisfi l'invariante (astratto) che il valor medio delle coordinate X e il valore medio delle coordinate Y approssimino i valori delle coordinate del valore ottenuto mediante `getBaricentro()` entro il valore di precisione dato da `getPrecisione()`. Specificare formalmente tale invariante astratto.
2. Si specifichino i metodi inserisciPunto e eliminaPunto in modo tale da assicurare che la proprietà invariante sia soddisfatta.
3. Si ipotizzi che l'invariante astratto venga ulteriormente rafforzato richiedendo che i punti della nuvola debbano avere coordinate non negative (non è richiesto scrivere questa nuova versione dell'invariante). Si ipotizzi anche che l'implementazione di NuvolaPunti venga realizzata mediante un ArrayList. Quale rep invariant deve essere rispettato dagli oggetti concreti della classe NuvolaPunti?

SOLUZIONE

```
Denotiamo con sommaX l'espressione (\sum Punto pun; isIn(pun); pun.getX())
e con sommaY l'espressione (\sum Punto pun; isIn(pun); pun.getY()).
1)

//@ public invariant numPunti()>0 ==>
//@  Math.abs(sommaX/numPunti() -getBaricentro.getX()) <=getPrecisione() &&
//@  Math.abs(sommaY/numPunti() -getBaricentro.getY()) <=getPrecisione();
```

2) La specifica di inserisciPunto deve garantire l'inserimento di p e la proprieta' invariante. Per garantire che il metodo non costruisca una nuvola che viola l'invariante, occorre specificare nella signals che l'insieme dei punti attuali uniti al nuovo punto p violerebbe l'invariante.

```
//@ requires p != null;
//@ ensures
//@  (\forall Punto pun;; isIn(pun) <==> \old(isIn(pun)) || pun == p) &&
//@  Math.abs(sommaX/numPunti()-getBaricentro.getX()) <=getPrecisione() &&
//@  Math.abs(sommaY/numPunti() -getBaricentro.getY()) <=getPrecisione();
//@ signals !\old(isIn(p) && (\forall Punto pun;; isIn(pun) <==> \old(isIn(pun)) ) &&
//@  Math.abs((p.getX()+ sommaX)/(numPunti()+1)-getBaricentro.getX())>getPrecisione() &&
//@  Math.abs((p.getY()+ sommaY)/(numPunti()+1)-getBaricentro.getY())>getPrecisione();
public void inserisciPunto(Punto p);
```

La estraiPunto è analoga, ma la signals deve considerare la villazioen dell'invariante nel caso in cui p sia eliminato.

```
//@ requires p != null;
//@ ensures
//@  (\forall Punto pun;; isIn(pun) <==> \old(isIn(pun)) && pun != p) &&
//@  Math.abs(sommaX/numPunti()-getBaricentro.getX()) <=getPrecisione() &&
//@  Math.abs(sommaY/numPunti() -getBaricentro.getY()) <=getPrecisione();
//@ signals \old(isIn(p) && (\forall Punto pun;; isIn(pun) <==> \old(isIn(pun)) ) &&
//@  Math.abs((-p.getX()+ sommaX)/(numPunti()-1)-getBaricentro.getX())>getPrecisione() &&
//@  Math.abs((-p.getY()+ sommaY)/(numPunti()-1)-getBaricentro.getY())>getPrecisione();
public void estraiPunto(Punto p);

3)
//rep:
private ArrayList<Punto> punti:
//@ private invariant punti!=null && !punti.contains(null) &&
//@  (forall int i; 0<=i && i<punti.size(); punti.getX()>=0 && punti.getY()>=0);
```

Esercizio 4

Si consideri il seguente metodo Java:

```
public static void testMethod(int x, String s) {
   int l;

   if (x < 0) return;
   l = s.length();

   while (x >= 2) {
      if (x >= 0 || l > 0) x = x % l;
         else break;
   }
   return;
}
```

1. Si disegni il diagramma del flusso di controllo;
2. Si identifichi, se esiste, un insieme di test (minimo) per coprire tutte le istruzioni e le decisioni (branch) del metodo.
3. Si identifichi, se esiste, un insieme di test (minimo) per coprire tutte le istruzioni e le condizioni del metodo.
4. In generale un insieme di test che copre tutte le decisioni compre anche tutte le condizioni? Nel caso non fosse vero, potremmo dire che invece un insieme di test che copre tutte le condizioni compre anche tutte le decisioni?

1) ...

2) Non esiste un insieme che possa coprire tutte le istruzioni: il break (e il relativo branch) non è raggiungibile (infatti, il resto della divisione di un numero positivo x per un intero l non può che essere non negativo). Un insieme che copre tutti le rimanenti istruzioni e branch è costituito da due dati di test:

Primo dato: per uscire dal primo return basta prendere $x < 0$, s qualsiasi;

Secondo dato: per entrare nel loop e poi uscire dal secondo return, occorre che: $x \geq 2$, con s diverso da null (per evitare eccezione all'istruzione $l = s.length$) e di lunghezza tale per cui $x\%s.length < 2$ (p.es. $x = 2, s.length = 2$, e quindi il resto è zero.) Infatti, se si scegliesse un s per cui $x\%s.length \geq 2$, si entrerebbe in un loop infinito (non raggiungendo mai il return).

3) E' impossibile rendere falsa la condizione $x \geq 0$: la condizione non può quindi essere esercitata. Di fatto, il valore della condizione $l > 0$ (a causa della valutazione corto-circuitata), non è preso in considerazione nella valutazione del valore dell'or. Si può esercitare il caso false della condizione $l > 0$, che corrisponde a $l = 0$, ossia un dato di test del tipo $(2, "")$, che causa tuttavia una DivisionByZeroException.

4) Naturalmente coprire tutte le decisioni non garantisce la copertura della condizioni. Anche il viceversa, tuttavia, non vale in generale: si tratta di casi "patologici", in cui il valore di verita' di una condizione non cambia anche assegnando tutti i possibili di valori di verita' alle condizioni elementari di cui e' composta. Esempi tipici sono i seguenti, dove x e y sono variabili booleane:

```
if (x && y && !x)) ...//sempre falso
else ...
if (x&&y ? !x : false) ... //sempre falso
else ...
if (foo(x && y)) ...//dove foo restituisce sempre false
else ...
```

Pur esercitando tutte le condizioni per x e per y, i branch else non possono essere eseguiti.

Esercizio 5

Si consideri la classe `Stack` che implementa la seguente interfaccia:

```
interface Stack_interface {
   void push(int n);
   int pop();//estrae e ritorna l'elemento in cima allo stack
   boolean is_empty();
}
```

Si ipotizzi che oggetti della classe `Stack` possano essere manipolati concorrentemente e che quindi si debba garantire un accesso corretto. Se un task cerca di effettuare l'operazione `pop` quando l'oggetto stack è vuoto, esso dovrà essere sospeso fino a che un altro task non avrà effettuato un'operazione `push` (a meno che non intervenga una `InterruptedException`).

Si completi la parziale implementazione dei metodi in modo da garantire appunto un accesso corretto.

```
class Stack implements Stack_interface {
   private class Element {
      int info;
      Element next;
      Element(int n, Element e) { info = n; next = e; }
   }
   private Element first;
   public Stack() { first = null; }

   public void  push(int n){



   }
   public             int pop(){


   }
   public              boolean is_empty(){


   }
}
```

SOLUZIONE Una soluzione, senza particolari requisiti di efficienza, consiste nel dichiarare synchronized i tre metodi. La push deve notificare i thread che sono in attesa che la pila non sia vuota per completare un pop. Ignoriamo anche la gestione di InterruptedException.

```
   public synchronized void push(int n){
         first = new Element(n,first);
         notifyAll();
   }
   public synchronized int pop(){
         while (first == null) wait();
         Element elm = first;
         first = first.next;
         return elm.info;
   }
   public synchronized boolean is_empty(){
         return (first == null);
  }
```